Task 1:

```
INCLUDE Irvine32.inc
.386
.MODEL flat, stdcall
.STACK 4096

.DATA
	msg  BYTE "Enter a value to push: ",0
	msg1 BYTE "Popped Value: ",0

.CODE
main PROC

	mov edx, offset msg
	call writestring
	call readint
	push eax

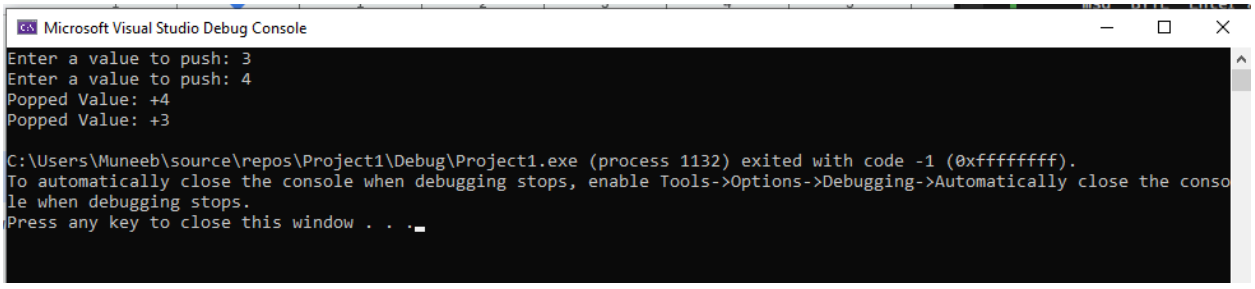
	call writestring
	call readint
	push eax

	mov edx, offset msg1
	call writestring
	pop eax
	call writeint
	call crlf

	call writestring
	pop eax
	call writeint
	call crlf

main ENDP


END main
```



Task 2:

```
INCLUDE Irvine32.inc
.386
.MODEL flat, stdcall
.STACK 4096

.DATA
	msg  BYTE "Enter a value to push: ",0
	msg1 BYTE "Popped Value: ",0

.CODE

push_value PROC
	pop ebp
	mov edx, offset msg
	call writestring
	call readint
	push eax
	push ebp
	ret
push_value ENDP

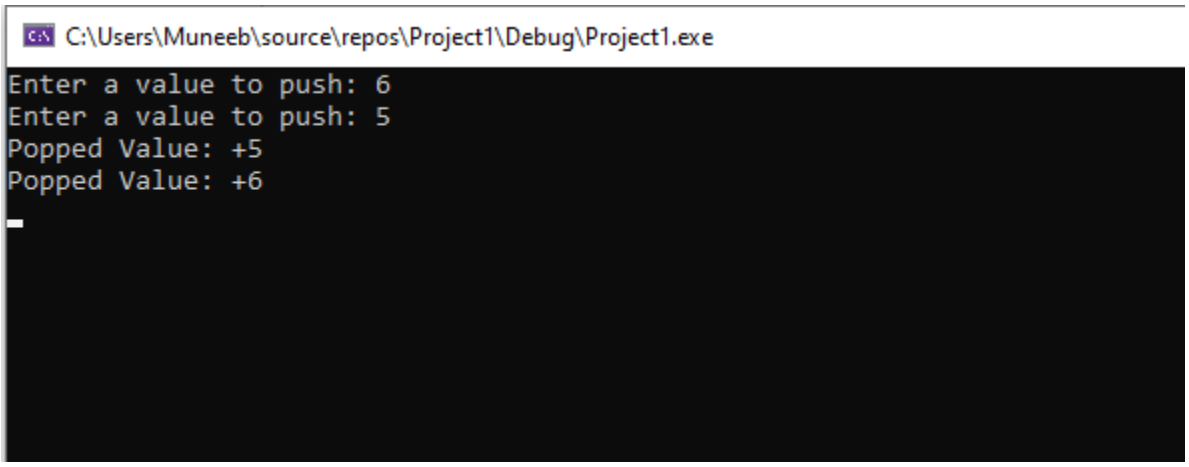
pop_value PROC
	pop ebp
	pop eax
	push ebp
	ret
pop_value ENDP

print_result PROC
	pop ebp
	mov edx, offset msg1
	call writestring
	call writeint
	call crlf
	push ebp
	ret
print_result ENDP

main PROC
	call push_value
	call push_value
	call pop_value
```

```
        call print_result
        call pop_value
        call print_result
main ENDP

END main
```



Task 3:

```
INCLUDE Irvine32.inc
.386
.MODEL flat, stdcall
.STACK 4096

.DATA
        msg byte "Reverse this string"
        strSize dd ?

.CODE
main PROC

mov edx, offset msg
call writestring
call crlf

mov strSize, lengthof msg
mov ecx, strSize

mov esi, offset msg

pushLoop:
        push [esi]
```

```
        inc esi
loop pushLoop

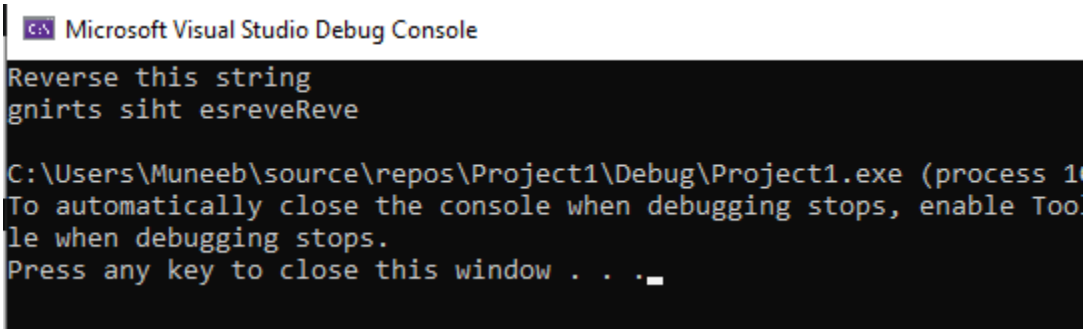
mov ecx, strSize
mov esi, offset msg

popLoop:
        pop [esi]
        inc esi
loop popLoop

mov edx, offset msg
call writestring
call crlf

exit
main ENDP

END main
```



Task 4:

```
INCLUDE Irvine32.inc
.386
.MODEL flat, stdcall
.STACK 4096

.DATA
        prompt   BYTE "Enter a string of up to 50 characters: ", 0
        msg     BYTE 50 DUP (?), 0
        strSize  DWORD 0
        msg2   BYTE "Reversed: ", 0

.CODE
main PROC
```

```
        mov edx, OFFSET prompt
        call writestring
        call crlf

        mov edi, OFFSET msg
        mov ecx, 50
        xor eax, eax

inputLoop:
        call readchar
        cmp al, 0Dh
        je endInput
        call writechar
        mov [edi], al
        inc edi
        loop inputLoop

endInput:
        mov BYTE PTR [edi], 0
        sub edi, OFFSET msg
        mov strSize, edi

        mov esi, OFFSET msg
        mov ecx, strSize

pushLoop:
        movzx eax, BYTE PTR [esi]
        push eax
        inc esi
        loop pushLoop

        mov esi, OFFSET msg
        mov ecx, strSize
        mov edx, OFFSET msg2
        call crlf
        call writestring

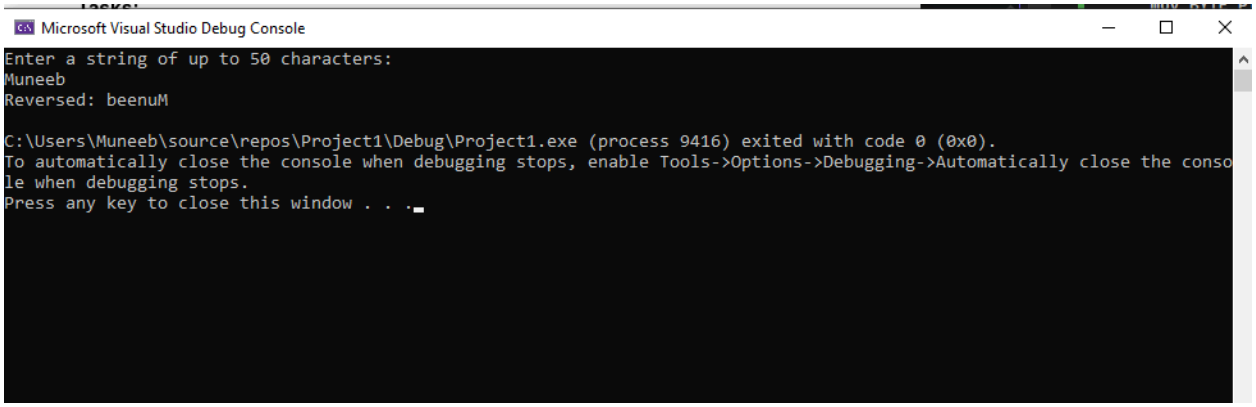
popLoop:
        pop eax
        mov [esi], al
        inc esi
        loop popLoop

        mov edx, OFFSET msg
```

```
        call writestring
        call crlf

        exit
main ENDP

END main
```



Task 5:

```
INCLUDE Irvine32.inc
.386
.MODEL flat, stdcall
.STACK 4096

.DATA
        arr dd 3 DUP(?)
        arrSize dd 3
        capacity dd 0
        menu byte "Choose an option: ",0
        op1 byte "1. Push",0
        op2 byte "2. Pop",0
        op3 byte "3. Display",0
        op4 byte "4. Exit",0
        pushPrompt byte "Enter Value to Push: ",0
        popMsg byte "Popped Value: ",0
        displayMsg byte "Current Stack State: ",0
        ovError byte "Overflow",0
        uvError byte "Underflow",0
        exitMsg byte "Exiting",0
        invalidMsg byte "Invalid choice! Try again.", 0
        space byte " ",0
        emptyMsg byte "The Stack is Empty",0
        choice dd 0
```

```
        val dd 0

.CODE

menuDisplay PROC
        mov edx, OFFSET menu
        call writestring
        call CRLF
        mov edx, OFFSET op1
        call writestring
        call CRLF
        mov edx, OFFSET op2
        call writestring
        call CRLF
        mov edx, OFFSET op3
        call writestring
        call CRLF
        mov edx, OFFSET op4
        call writestring
        call CRLF
        ret
menuDisplay ENDP

pushFunc PROC

        pop ebp
        mov ecx, capacity
        cmp ecx, arrSize
        jae overflowed
        mov edx, offset pushPrompt
        call writestring
        call readint
        mov val, eax
        mov [edi], eax
        add edi,4
        inc ecx
        mov capacity,ecx
        jmp endPush

        overflowed:
        mov edx, offset ovError
        call writestring
        call crlf
```

```
        endPush:
        push ebp
        ret
pushFunc ENDP

popFunc PROC

        pop ebp
        mov ecx,capacity
        cmp capacity,0
        JBE underflowed
        sub edi, 4
        dec ecx
        mov capacity, ecx
        jmp endPop


        underflowed:
        mov edx, offset uvError
        call writestring
        call crlf
        endPop:
        push ebp
        ret
popFunc ENDP

displayFunc PROC
        pop ebp

        mov ecx, capacity
        cmp ecx, 0
        jbe emptyStack
        mov esi, offset arr
        mov edx, offset displayMsg
        call writestring

        displayLoop:
        mov eax, [esi]
        add esi,4
        mov edx, offset space
        call writestring
        call writeint

        loop displayLoop
```

```
        call crlf
        jmp endDisplay

        emptyStack:

        mov edx, offset emptyMsg
        call writestring

        call crlf

        endDisplay:
        push ebp
        ret
displayFunc ENDP

main PROC
        mov edi, OFFSET arr
        mov ecx, arrSize

startProgram:
        call menuDisplay
        call readint
        mov choice, eax         ; Store user input

        cmp choice, 1
        je pushOption
        cmp choice, 2
        je popOption
        cmp choice, 3
        je displayOption
        cmp choice, 4
        je exitProgram

        call crlf
        mov edx, OFFSET invalidMsg
        call writestring
        jmp startProgram

pushOption:
        call pushFunc
        jmp startProgram

popOption:
```

```
        call popFunc
        jmp startProgram

displayOption:
        call displayFunc
        jmp startProgram

exitProgram:
        call crlf
        mov edx, OFFSET exitMsg
        call writestring
        exit
main ENDP
END main
```

